

박스 1개가 도착해서 팔레트에 놓이기까지 — 동한 파트

흐름도 1~8단계 중 동한이 맡은 부분과 오늘 바뀐 부분을 정리했습니다. 동한 파트는 1단계 탐뷰 깊이 카메라 박스 상태 인식(신규), 2단계 재인식 규정(제안), 5-③~⑥ 자리 순위·미래 평가, 7단계 Gazebo + MoveIt 물리 실행입니다. 다른 사람 파트는 동한 파트와 맞닿는 부분만 적었습니다.

태현 님 문서의 N = 5 탐색, 평평한 층 규칙, `config/taehyeon/environment.yaml` 은 아직 통합 저장소에 없습니다. 그래서 그 부분은 동한 파트와의 접점만 적었습니다. 수치는 모두 이 저장소 코드(main e7b3c06과 이번 PR)로 2026-10-09에 직접 실행한 결과입니다.

변경 · 신규 오늘 바뀌거나 새로 생긴 부분 **제안** 코드는 있지만 런타임에 아직 연결 안 됨 **유지** 기존 그대로

다른 파트 접점만 표시

0

환경 설정 정리 필요

동한 설정 파일은 모두 `config/donghan/` 에 있습니다.

파일	내용
<code>perception_depth.yaml</code>	1단계 깊이 측정·SKU 추정: 집는 구역(ROI), 높임 12 mm, 기울기 3°, 모서리 20 mm, 유효 깊이 85 %, 허용오차 12 mm / 4 % / 8 %
<code>reinspection_policy.yaml</code>	2단계 재인식: 재시도 상한, 박스당 시간 예산 (모두 가정치)
<code>candidate_runtime_v2.yaml</code> , <code>model_*.yaml</code>	5-③~⑥ 플래너: Top-K, 시나리오 수, 미래 단계 수(horizon), 점수 가중치

태현 님 원칙(현장 값은 `environment.yaml` 한 곳)에 맞추려면 `perception_depth.yaml` 의 집는 구역과 컨베이어 좌표계를 그 파일로 옮겨야 합니다. 좌표계는 픽업 구역 중심 (-1.08, 1.20), 롤러 윗면 z 0.895 m로 둔 가정치입니다. 임계값과 가중치는 알고리즘 설정이라 동한 파일에 남깁니다.

1

인식 — 탐뷰 카메라 박스 상태 인식 모델 신규

집는 위치(스토퍼)에 멈춘 박스를 위에서 내려다보는 RGB-D 깊이 영상으로 캡니다. 결과로 크기·자세·파손 단서를 만들고, 2단계가 그대로 받는 `RawObservation` 으로 넘깁니다. 인식은 관측만 만들고 이상 판정은 2단계가 합니다.

저울 무게 (정지 계량)

ID·라벨 판독 (못 읽으면 None)

탐뷰 깊이 영상 → `measure_box`

라벨 못 읽음 → `resolve_sku` (크기 + 무게)



`build_raw_observation`



`RawObservation` (+ 원신호)



2단계 `StateValidator`

measure_box: 깊이 영상 한 장 → 박스 상태

- 1 깊이 영상을 컨베이어 좌표 점군으로 바꿉니다. 0.2~4 m 안의 유효 깊이만 씁니다.
- 2 집는 구역(ROI) 안에서 롤러보다 3 cm 이상 높은 점을 박스로 봅니다.
- 3 **대기 박스 분리**: 컨베이어 방향으로 2 cm 이상 틈이 있으면 다른 박스로 나눕니다. 스톱퍼 쪽 박스를 재고, 보이는 박스 수와 앞뒤 간격도 남깁니다. 틈 없이 붙은 두 박스는 하나로 보이며, 2단계 재인식이 잡습니다(RIC5).
- 4 **크기·자세**: 볼록 껍질에서 최소 면적 사각형을 찾아 긴 변·짧은 변·yaw를 구합니다. 높이는 윗면 높이의 상위 90 % 값이라 작은 높임이나 가장자리 잡음에 흔들리지 않습니다.
- 5 **파손 단서**: 윗면 눌림 12 mm 초과, 기울어짐 3° 초과, 모서리 찌그러짐(사각형 채움 0.92 미만 또는 모서리 빈 거리 20 mm 초과). 하나라도 있으면 `visual_damage = True` 입니다. 구역 경계에 걸린 박스는 크기가 하한값이라 파손 판단을 생략합니다(`in_full_view = False`).
- 6 **확신도**: 유효 깊이가 85 % 미만이거나 시야 경계에 걸리면 0.6(태현 님 `uncertain_confidence` 와 같은 값)이 되고, 2단계에서 측정 불확실성 δ 로 이어집니다.

resolve_sku: 라벨을 못 읽었을 때만

- 크기는 축별 $\pm \max(12 \text{ mm}, 4 \%)$, 무게는 SKU 범위 $\pm 8 \%$ 안이면 후보로 봅니다. 2단계 검증과 같은 기준이라 추정이 맞으면 2단계도 통과합니다.
- 후보가 정확히 1개일 때만** SKU를 정하고 확신도를 0.9로 둡니다. 2단계는 이 박스를 불확실(δ) 박스로 다룹니다.
- 후보가 여러이거나 없으면 라벨 없음으로 넘깁니다. 이후 Base-view를 거쳐 Inspection으로 갑니다. 찍어서 틀리게 하지 않습니다.

출력

RawObservation (태현 님 계약 그대로)	PerceptionSignals (2단계 재인식용 원신호)
box_id, label_sku, weight_kg, size(긴 변·짧은 변·높이), confidence, visual_damage, view = "top", stamp	in_full_view, valid_ratio, box_count, 도착 yaw, damage_score, 프레임별 크기, 저울 안정·영점

확인한 것

- 합성 깊이 영상 테스트와 2단계 계약 테스트: `tests/donghan/test_depth_measurement.py` , `test_sku_resolver.py` , `test_stage1_to_stage2_contract.py`
- 통합 실행**(80박스 주문 `example_order.json` , 깊이 영상 → ... → 실행 계약까지): 라벨 없는 박스 66개 중 58개를 맞게 추정했고, 8개는 '모호'로 Inspection에 갑니다. **잘못된 SKU로 정한 경우는 0개**입니다.
- 모호 8개는 모두 K07($0.34 \times 0.26 \times 0.15 \text{ m}$)과 K09($0.35 \times 0.25 \times 0.15 \text{ m}$)입니다. 두 SKU는 크기 차이가 허용 오차 안이고 무게 범위(0.8~10 kg)도 같아서, 크기·무게로는 원리적으로 구분할 수 없습니다. **이 두 SKU는 라벨 판독이 꼭 필요합니다.**
- 측정 1회 약 26 ms (320×240 합성 영상, 개발 PC)
- 브라우저 시뮬레이터 `viewer/donghan/perception_sim.html` : 초입 카메라 유무 비교 기능이 있습니다 (demo_original 8×6, 붙어서 투입 30 %에서 판정 오류 0 vs 8, 두 박스가 함께 계량된 경우 0 vs 19).

아직 안 된 것

- 실제 깊이 카메라가 없습니다. Gazebo 카메라는 RGB입니다(`config/camera.yaml` `optional_3d_camera: false`).
- 이 모델로 `/pac/observation` 을 내보내는 ROS 노드가 아직 없습니다. 지금은 계산부만 있습니다.
- 컨베이어 좌표계 원점과 짐의 구역은 가정치입니다(재성 님 확인 필요).

접점 · 태현 님 1단계 "다음 5개 박스"

태현 님 탐색은 컨베이어 위 다음 5개 박스의 크기·무게·순서를 씁니다. 이 모델은 짐의 구역(길이 0.73 m) 안의 대기 박스를 나눠서 크기를 썰 수 있습니다(`target_x`). 하지만 구역에는 1~2개만 들어오고, 무게는 저울을 지나야 합니다. 5개를 보려면 초입 카메라나 더 넓은 시야가 필요하고, 그 박스들의 무게는 SKU 기준값으로 채워야 합니다. **어떻게 할지 결정이 필요합니다.**

2

상태 검증 유지 + 재인식 제안

태현 님 `StateValidator` 를 그대로 씁니다. 태현 님 문서의 측정 오차 $\pm 1 \text{ mm}$ / $\pm 1 \%$ 는 인식 잡음 모델 (`PerceptionConfig`)이고, 2단계 판정 허용오차는 12 mm / 4% / 무게 8% 입니다. 1단계 SKU 추정도 같은 허용오차를 씁니다.

재인식 RI 코드 — `ros2_ws/src/pac_reinspection/` (동한, 제안)

`ReinspectionValidator` 가 `StateValidator` 를 고치지 않고 감쌉니다. `validate` 시그니처가 같아서 그대로 바꿔 끼울 수 있습니다.

RawObservation (+ 원신호)

- 사전 검사: 저울 미안정·영점 이상, 라벨 체크섬, 프레임 간 크기 분산, 시야 경계, 깊이 결측, 두 박스 합쳐짐, 도착 자세
- 필요하면 1단계에 재판독·재계량·재촬영 요청
(상한: 라벨 2회 · 계량 2회 · 촬영 3회, 박스당 20 s — 가정치)
- `StateValidator.validate` (그대로)
- 사후 처리: 미등록 SKU·무게·높이 불일치를 다시 감지해 재확인
- 정상 → PLAN
- 센서 의심 미해소 → PLAN + δ , SENSOR_UNCERTAIN
- 두 박스 합쳐짐·자세 이상·남은 수량 0인 SKU·판정 반복 → INSPECTION

- 코드는 A 신원, B 센서 교차, C 인식 품질, D 외관, E 흐름, F 작업 중으로 나뉘었습니다. 세부는 `docs/donghan/reinspection_policy.md` 에 있습니다.
- 기존 이상 종류 5개, 경로, `RejectCode` 는 늘리지 않았습니다. 세부 사유는 `Verdict.notes` 에 "RI-xxx: ..." 로 남깁니다.
- 교체 시험(`tools/donghan/reinspection_drop_in.py`)에서 기존 런타임 테스트 21개가 모두 통과했습니다. 오늘 80박스 통합 실행에서도 기본 검증기와 결과가 같았습니다.
- **연결 안 됨:** `pac_runtime/core.py` 는 아직 `StateValidator` 를 씁니다. 태현 님 확인 후 연결합니다.

주의: 박스당 재인식 예산 20 s(가정치)는 도착 간격 6 s, 결정 시간 2 s보다 훨씬 깁니다. 재인식이 생기면 컨베이어가 기다리게 되므로, 태현 님 환경 값에 맞춰 다시 정해야 합니다.

팔레트 교체, 컨베이어 대기, 이상 상황 처리(태현). 동한 쪽에는 "대기열에서 본 박스 수가 남은 기대 수보다 많으면 재집계" 비교 함수(`inventory_consistency` , RI-E3)만 제안으로 있습니다.

놓기 / 버퍼에 두기 / 버퍼 박스 꺼내기는 태현 님이 정합니다. 각 선택지의 자리를 물을 때 쓰는 자리 선택기(placer)에 동한 순위를 끼울 수 있습니다(`ranker:=donghan`).

- 지금 코드에서는 선택지마다 동한 순위가 1번씩 돕니다. 순위 1회(로봇 사전 검사 포함)는 개발 PC 기준 중앙값 약 1.8~2.0 s입니다. 5단계 "남은 문제"를 참고하세요.
- 태현 님 N = 5 탐색은 팔레트 복사본마다 여러 박스를 쌓아 봅니다. 그래서 **탐색 안에서는 평평한 층 규칙을, 실제로 실행할 자리에만 동한 순위를 쓰는 구성**이 맞습니다(제안). 탐색 코드가 통합되면 이 경계를 같이 정해야 합니다.

5-① 후보 생성 · 5-② 안전 검사 (태현, 유지)

빈 공간 모서리(EMS) + 기존 박스 꼭짓점(EP), 8 cm 이내 합침. 지지율, LBCP, 하중, 높이·겹침·경계 검사. **동한 파트는 이 결과(안전 후보)만 받고, 안전 검사를 바꾸거나 건너뛰지 않습니다.**

기본 순서 vs 동한 순위

기본은 태현 님 순서입니다(DBLF, 새 문서에서는 평평한 층 규칙). `ranker:=donghan` 일 때 이 자리에 동한 ③~⑥이 들어갑니다. 기본값은 계속 `dblf` 입니다.

5-③~⑥ 동한 플래너 — `ros2_ws/src/pac_planning/`

- ③ **Feature 계산** — 후보마다 45개를 계산합니다. OPAL 15개(위치·크기, 지지율·지지 중심, 높이, 하중비, 측면 지지, 작업량)에 다음 30개를 더합니다: 정규화 좌표·크기, 잔여재고 적합도, 버퍼 적합도·점유, 의존성(밀에 깔린 박스 수), 로봇 접근 여유, 무게중심·높이 무게중심 이동, 하중 증가, 압축도, 가장 큰 빈 면, 조각남, 높이 변화, 안전 포화, 하중 여유, 균형, 시간 비용, EMS 여부, 미리 본 비율, 남은 부피 비율, 센서 불확실(δ).
- ④ **AI 평가 → Top-K** — NumPy MLP(`DualHeadRanker`)가 두 가지를 냅니다. 하나는 순위 헤드(LambdaRank, 한 결정 안에서만 비교)이고, 다른 하나는 미래값 헤드 4개(평균, 하위 20 % 평균 CVaR, 막힘률, 실패율)입니다. 모델이 없거나 입력이 학습 분포 밖이면 현재 점수로 정렬합니다. 정렬 순서는 안전 목표 충족 → 안전도 → 점수이고, **상위 4개(Top-K = 4)**만 다음 단계로 갑니다.
- ⑤ **Top-K 미래 롤아웃** — 남은 주문 비율로 미래 박스를 뽑아 시나리오 7종을 만듭니다(무작위, 큰 박스 나중, 무거운 박스 나중, 작은 박스 먼저, 같은 SKU 연속, 조각남, 스트레스). horizon은 3입니다. Top-K 모두 **같은 시나리오**로 greedy 배치해 공정하게 비교합니다. 시간 예산은 1 s입니다. 남은 시간이 60 % 미만이면 Top-K를, 35 % 미만이면 시나리오 수를 절반으로 줄이고, 20 % 미만이면 horizon을 1로 줄입니다. 안전 검사는 줄이지 않습니다.
- ⑥ **최종 점수 → 최종 후보**

$$\text{점수} = 0.15 \times \text{안전(포화)} + 0.3 \times \text{공간 품질} + 0.4 \times \text{미래 가치(평균)}$$

$$- 0.1 \times \text{위험} - 0.05 \times \text{시간}$$

$$\text{위험} = \max(0, \text{평균} - \text{CVaR}) + 0.25 \times \text{막힘률} + 0.25 \times \text{실패율}$$

$$\text{롤아웃을 못 한 후보의 미래 가치} = \text{AI 예측값 (모델이 없으면 0)}$$

모델 상태(유지): 학습 후보 모델 `candidate_runtime_v2` 는 360 episode 비교에서 DBLF를 이기지 못했습니다(정상 조건 평균 팔레트 수 DBLF 2.389 vs 학습 2.611). 그래서 기본값은 모델 없이(휴리스틱 + 롤아웃)입니다. `models/dual_head_ranker.json` 은 참조 후보기로 학습한 모델이라, 실제 EMS 후보기에서는 검증되지 않았습니다.

오늘 바뀐 것 (PR #1, main d66aa73)

- ROS `runtime_node` · `verified_runtime` 와 `launch` 2개에 `ranker:=dblf|donghan` , `ranker_model_path` , `ranker_config` , `ranker_seed` 를 추가했습니다. 전에는 ROS로 실행하면 ③~⑥ 없이 DBLF로만 돌았습니다.
- 순위 연결 코드 4곳을 `pac_planning.team_bridge` 1곳으로 합쳤습니다.
- 학습 모델과 플래너 설정(`horizon`, 시나리오 수, `CVaR`)이 다르면 시작할 때 오류가 납니다. 전에는 매 호출 조용히 휴리스틱으로 대체했습니다.
- 재성 님 V4.4 브리지가 저장소 밖 옛 폴더 대신 통합 저장소의 플래너를 씁니다.

발견하고 고친 문제 — 로봇이 갈 수 있는 후보 안에서만 순위 (이번 PR, 커밋 2743e62)

동한 순위는 6단계(로봇 실행 가능성)를 모르고 자리를 골랐습니다. 게다가 후보를 Top-K 4개만 넘겨서 "떨어지면 다음 후보로"도 4번까지만 댔습니다. 그래서 동한 순위가 고른 자리는 로봇이 못 가는 경우가 많았습니다. 이제는 `RobotAwarePlacer` 가 순위에 로봇 검사기를 넘기고(`bind_robot`), 순위는 **로봇이 갈 수 있는 후보만** 받아 매깁니다. 안전 검사(5-②)와 로봇 검사(6)는 그대로입니다.

실행 가능한 자리가 없던 선택지 / 팔레트 수	DBLF	동한 (수정 전)	동한 (수정 후)
시드 1	3 / 6	40 / 7	10 / 6
시드 2	10 / 5	62 / 6	4 / 5

조건: Gazebo 로봇 설정(`robot_check_gazebo.yaml`), 80박스 주문(`example_order.json`), 가벼운 플래너 설정(시나리오 1, `horizon` 1), 80박스 모두 적재. 수정 후 6단계 거부는 0회입니다. 팔레트 수는 DBLF와 같고, 버퍼에 두기·꺼내기가 DBLF보다 절반 정도로 줄었습니다(시드 1: 34 → 16회). 표본은 주문 1개 × 시드 2개라 작아서, DBLF보다 낫다고 결론 내릴 수는 없습니다. (수정 전 Top-4 뒤에 DBLF 순서를 붙여 보는 시도는 시드 1에서만 도움이 됐습니다.)

남은 문제 · 결정 시간 (동한)

런타임에서 플래너 시간 예산(1 s)을 컷고(`donghan_ranker(use_time_budget=True)`), Top-K·시나리오·`horizon`을 줄이는 단계도 실제로 동작합니다. 그래도 순위 1회는 중앙값 약 1.8~2.0 s(p90 3~3.9 s), 결정 1회당 약 3.6 s입니다(25박스, 개발 PC). 시간 예산이 롤아웃에만 걸리고, 특징 계산과 로봇 사전 검사(후보당 약 62 ms × 중앙값 12.5개 ≈ 0.8 s)에는 걸리지 않기 때문입니다. **태현 님 결정 시간 2 s 안에 들어오지 않으므로, 실제 셀은 당분간 기본값 dblf 로 운영합니다.** 다음 작업은 특징 계산과 로봇 검사 결과를 같은 상태 버전 안에서 재사용(캐시)하는 것입니다.

HDR50-22 IK, 하강 경로 · 팔 충돌, 가반하중(태현). 순위 목록 앞에서부터 최대 12개를 검사합니다 (`robot_checks_per_option = 12`). 동한 순위는 이 검사를 먼저 통과한 후보만 순위를 매깁니다(이번 PR). 그래서 순위 1등이 6단계에서 떨어지지 않습니다. 7단계 실행부는 6단계가 고른 흡착 방향(`gripper_yaw_rad`)을 그대로 따릅니다.

`ros2_ws/src/pac_execution/` (동한)

- **verified_runtime**: 태현 님 런타임(`CoreBridge`)을 그대로 쓰되 결과를 엄격하게 받습니다. 명령 버전 · 박스 · 후보 ID가 일치해야 하고, 측정된 위치(Gazebo 월드 pose 또는 보정된 비전)만 커밋합니다. 외부 판정으로 런타임 검사를 덮어쓸 수 없고, 문제가 생기면 HOLD하고 `/pac/runtime_fault` 로 알립니다.
- **moveit_executor** 순서:

명령 검증 (오래된 계획, 팔레트 밖, 박스 + EOAT $15\text{ kg} \leq 50\text{ kg}$)

- `PRE_GRASP` → `DESCEND_GRASP` → 흡착 → `LIFT`
(박스가 실제로 따라 올라왔는지 12 mm 안으로 확인)
- `PRE_PLACE` → `DESCEND_PLACE` (지지면 13 mm 위에서 멈춤) → 놓기
- 안정된 위치 측정 → 이미 놓인 박스가 1 mm 넘게 움직였으면 중단
- `RETREAT` 후 다시 측정·검사 → `/pac/execution_result` 발행

- 그 밖에 `gazebo_grasp` (흡착 고정, C++), `gazebo_box_source` (원본 박스 투입), `launch original_boxes_demo.launch.py` 가 있습니다.
- 오늘: `verified_runtime` 이 import 단계에서 실패하던 문제(`make_runtime_ranker` 없음)를 고쳤습니다 (PR #1).

한계: 놓기(`PLACE_CURRENT`)만 지원합니다. 버퍼·NG·팔레트 교체·재적재 명령은 `CELL_ACTUATOR_REQUIRED` 로 멈춥니다. MoveIt/Gazebo 실행은 아직 해 보지 않았습니다(`gazebo_grasp` 는 Gazebo Fortress 개발 패키지가 없어 빌드하지 못함). 태현 님 재생 도구(`tools/realtime/*`)와는 별개입니다.

실제로 놓인 위치로 상태를 보정하는 것은 태현 님 코드 그대로입니다. 동한 쪽 변경(PR #1)은 다음과 같습니다.

- 로봇이 박스를 놓는 동안 도착한 관측과 컨베이어 유류 신호는 `CoreBridge.deferred` 에 보관했다가, 결과를 반영한 직후 처리합니다. 전에는 그 사이 상태 버전이 바뀌어 결과가 `STALE_RESULT` 로 거부되고 `runtime_node` 가 종료됐습니다(재현 확인).
- 거부된 메시지는 로그만 남기고 노드는 계속 돛니다. 거부된 결과는 위치를 임의로 확정하지 않습니다.

실시간 흐름 점점 확인 필요

태현 님 새 흐름은 "지금 박스가 계획대로 놓인다"고 가정하고, 로봇이 움직이는 동안 다음 박스를 미리 계산합니다. 지금 통합 코드는 안전을 위해 **결과가 온 뒤** 다음 박스를 계획합니다(위 8단계). 두 방식을 맞춰야 합니다.

로봇

박스 i 집기·놓기 (8 s)

박스 i+1

1단계 깊이 측정

i+1 측정 ≈ 0.03 s

지금 (통합 코드)

i+1 관측 보관 (deferred)

결과 후 계획

태현 님 새 흐름

i+1 미리 계산

결과 확인 → 사용 / 재계산

맞추는 방법(제안): (1) 계획대로 놓였다고 가정한 상태로 i+1을 미리 계산합니다. (2) 결과가 계획과 같으면(L0) 그 명령을 쓰고, 다르면(L1 이상) 실제 상태로 다시 계산합니다. 동한 쪽 시간은 깊이 측정 약 0.03 s, 순위 1회 약 1.8~2.0 s(플래너 예산 1 s 포함), 재인식 최대 20 s(가정치)입니다.

오늘 동한 파트 변경 요약

무엇	상태	내용
PR #1	main d66aa73에 병합	5-③~⑥ ROS 연결(<code>ranker</code> 파라미터), 순위 연결 1곳으로, 모델·설정 계약 검사, V4.4 브리지 경로, <code>verified_runtime</code> import 수정, 놓는 중 도착 관측 보관
PR #2	main e7b3c06에 병합	1단계 깊이 측정·SKU 추정, 2단계 재인식 RI 코드(제안), 브라우저 시뮬레이터
이번 PR (<code>feature/donghan-flow-doc-ranker-fix</code>)	리뷰 대기	<code>pac_perception</code> 의존성(<code>pac_runtime</code> , <code>numpy</code> , <code>yaml</code>) 선언 · 로봇이 갈 수 있는 후보만 순위 · 런타임 플래너 시간 예산 컴 · 이 문서
동한 순위 결정 시간	수정 필요	결정 1회 약 3.6 s > 2 s. 5단계 "남은 문제" 참고

확인이 필요한 점점

누구와	내용
태현 님	N = 5 탐색 안에서는 평평한 층 규칙, 실제 실행 자리에만 동한 순위 · 실시간 미리 계산과 <code>CoreBridge.deferred</code> 맞추기 · <code>ReinspectionValidator</code> 연결 여부와 재인식 시간 예산 · <code>environment.yaml</code> 로 옮길 현장 값
재성 님	컨베이어 좌표계 원점·집는 구역 · Gazebo 깊이 카메라(또는 초입 카메라) · 저울 안정·영점 신호 전달 · <code>pac_perception</code> 의존성 변경
팀	"다음 5개 박스" 무게를 어디서 얻을지(SKU 기준값?) · K07/K09처럼 크기·무게로 구분 안 되는 SKU의 라벨 판독

※ 수치 출처: 통합 저장소 main e7b3c06 + 이번 PR 코드, 2026-10-09 개발 PC(Windows, Python 3.11) 실행. 전체 테스트 417 통과(Windows 환경 문제 2 실패·12 오류 제외). MoveIt/Gazebo 실제 실행 결과는 포함하지 않았습니다.